

Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 14, 2026

Contents

1	Introduction	3
2	Quick Setup Guide	4
2.1	Environment Setup	4
2.2	Dependencies	4
3	Synthetic Data Generation Pipeline (data-gen)	5
3.1	File Structure Review	5
3.2	Explanation of the Engine and Scripts	5
3.2.1	data-gen/src/anomaly_generator.py	5
3.2.2	data-gen/script/generate_custom_dataset.py	5
3.2.3	data-gen/script/test_generator_deformation.py	6
3.2.4	data-gen/script/visualize_anomalies_templates.py	6
3.3	Anomaly Template Creation	6
3.3.1	Configuration Blueprint (CSV Logic)	6
4	1D Convolutional Neural Networks: Theoretical Background	9
4.1	The 1D Convolution Operation (Forward Pass)	9
4.1.1	Single Channel Convolution	9
4.1.2	Multi-Channel Convolution	9
4.2	Non-linear Activations	10
4.2.1	ReLU (Rectified Linear Unit)	10
4.2.2	Softmax	10
4.3	Pooling Layers (Downsampling)	10
4.4	The Loss Function	10
4.5	Backpropagation in 1D-CNNs (Backward Pass)	11
4.5.1	Backpropagating through MaxPooling	11
4.5.2	Backpropagating through 1D Convolution	11
4.6	Optimization and Regularization	12
4.6.1	Adam Optimizer	12
4.6.2	Regularization: Dropout and BatchNorm	12

5	1D-CNN Implementation	13
5.1	Data Processing (<code>train.py</code>)	13
5.2	Architecture Definition (<code>model.py</code>)	13
5.3	Training Loop (<code>train.py</code>)	14
5.3.1	Training Results	14
5.4	Architecture Search	15
5.5	Inference and Evaluation (<code>visualize_inference.py</code>)	16
5.5.1	Inference Results	16
5.6	Robustness Analysis (<code>robustness_test.py</code>)	17

1 Introduction

This document provides a comprehensive overview of the **Pole Anomaly Project**, an anomaly detection and classification system for nuclear gamma dosimetry signals. The overall objective is to automatically identify and classify transient anomalous events—such as radiation spikes with distinct geometric signatures—embedded within continuous, noisy gamma dose rate measurements.

The project follows a two-phase approach. First, a **synthetic data generation pipeline** produces large-scale training datasets by injecting user-defined anomaly templates (Bell curves, Squares, M-shapes, Exponential ramps, etc.) into realistic simulated radiation backgrounds derived from real-world sensor baselines. Second, a **1D Convolutional Neural Network (1D-CNN)** is trained on these synthetic datasets to perform multi-class anomaly classification in a streaming inference setting.

The repository is organized into three main components:

- **data-gen/**: The synthetic data generation pipeline, including anomaly templates, the stochastic generator engine, and visualization scripts.
- **1D-CNN/**: The neural network implementation, including the model architecture, training loop, inference engine, and robustness testing scripts.
- **documentation/**: This document and associated media.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.14 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install all dependencies
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following libraries:

- `numpy` & `pandas`: Fast numerical array manipulation and CSV template handling.
- `torch` (PyTorch): Deep learning framework for the 1D-CNN. Installed from the PyTorch Nightly channel with CUDA 12.8 support via `--extra-index-url`.
- `plotly` & `kaleido`: Interactive HTML visualizations and static PNG export.
- `scikit-learn`: Utility functions for metrics and data splitting.
- `matplotlib`: Supplementary plotting library.
- `scipy`, `statsmodels`, `tslearn`: Statistical interpolation and legacy dataset compatibility.

Note: The `--pre` flag in `requirements.txt` enables installation of pre-release PyTorch builds, which is required for CUDA 12.8 support on newer GPUs (e.g., NVIDIA RTX 5060).

3 Synthetic Data Generation Pipeline (data-gen)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data-gen/data/`: Contains raw background sensor readings (e.g., `2015_months_DebitDoseA.txt`) used as a real-world baseline, along with generated output datasets and the class mapping file (`class_mapping.txt`).
- `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for the 6 anomaly shapes: `bell`, `bell_sq`, `fast_ascend`, `fast_descend`, `M`, and `square`.
- `data-gen/src/`: Stores the pure mathematical generator library (`anomaly_generator.py`), decoupled from direct script logic.
- `data-gen/script/`: Stores execution pipelines and visualization launchers. Scripts utilize the math from `data-gen/src/` to generate datasets or debug plots.
- `data-gen/logs/`: The destination for Plotly’s interactive HTML visualizations and PNG exports.

3.2 Explanation of the Engine and Scripts

3.2.1 `data-gen/src/anomaly_generator.py`

This is the core computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian deformation using the limits declared in the CSV file.

3.2.2 `data-gen/script/generate_custom_dataset.py`

The primary execution pipeline for training data. It analyzes the original physical gamma sensor data using moving averages to dynamically separate high-frequency point-to-point noise from the slow, low-frequency sensor drift. It then simulates a highly realistic **wandering baseline** using a chunk-by-chunk Ornstein-Uhlenbeck (OU) mean-reverting random walk process, overlaying the high-frequency Gaussian noise on top.

Spanning datasets of up to 10,000,000 steps, it randomly samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and seamlessly injects them into the wandering baseline. It maps each injected shape to a unique multi-class integer ID, tagging these absolute ground-truth labels alongside the signal.

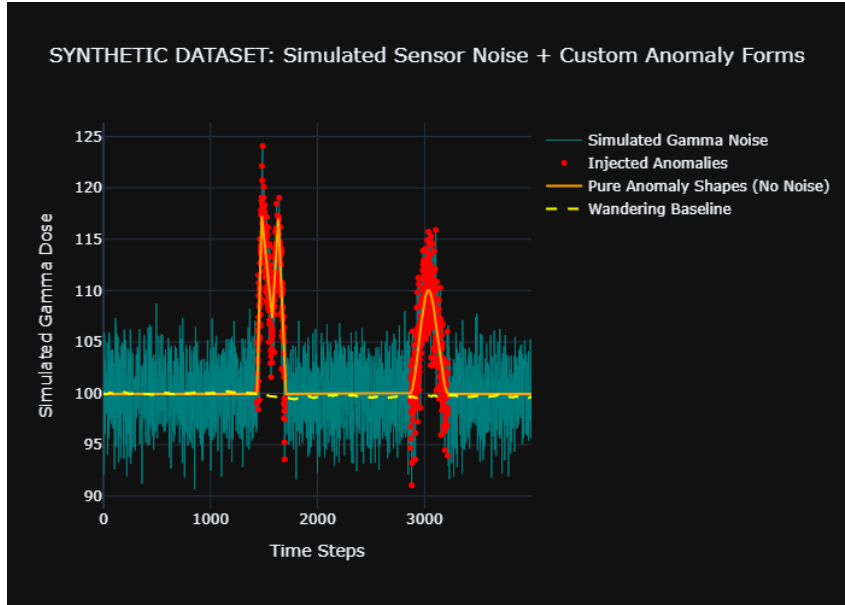


Figure 1: Output of `generate_custom_dataset.py`

3.2.3 `data-gen/script/test_generator_deformation.py`

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 generation iterations of each template to visually verify and validate the variability limits enforced by the template's deformation parameters. Exports directly to HTML and PNG.

3.2.4 `data-gen/script/visualize_anomalies_templates.py`

This script visualizes the pure original mathematical blueprints. It renders the shapes and their interpolation properties exactly as described in the blueprint CSV, without injecting randomization or temporal noise.

3.3 Anomaly Template Creation

Instead of relying on rigid pre-recorded dataset distributions, targeted geometric structures are freely defined as CSV blueprints in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping normalized coordinates and behavioral constraints:

`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value:** Normalized shape coordinates (scaling from 0.0 to 1.0). They define the key inflection points of the curve.
- **Deformation Multipliers (`t_minus`, `t_plus`, `v_minus`, `v_plus`):** Define spatial jitter limits per control point. A value of 1.0 allows symmetric Gaussian distortion up to 100% of the global variance parameter. A value of 0.0 locks the point rigidly in that direction.

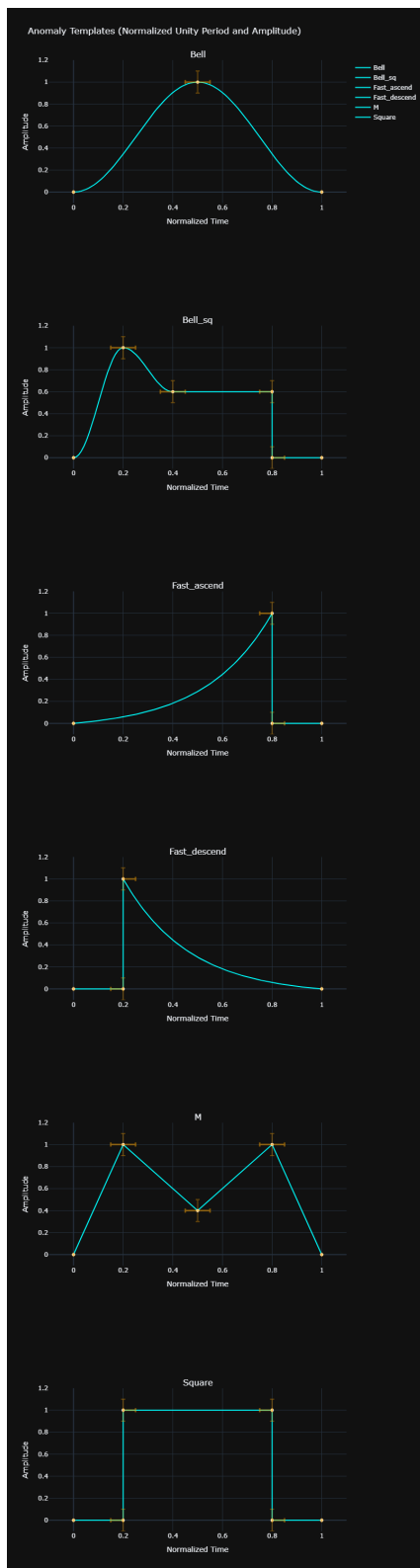


Figure 2: Output of visualize_anomalies_templates.py

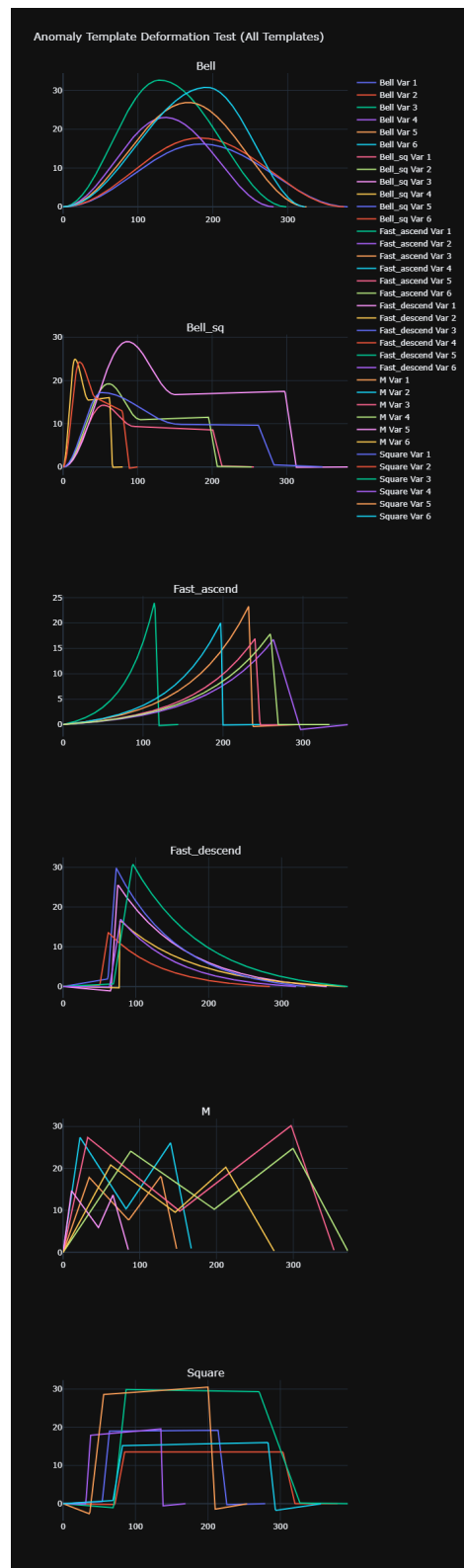


Figure 3: Output of test_generator_deformation.py

- **interp (Interpolation Mode):** Defines the mathematical connection between consecutive control points:
 - **linear:** Simple linear interpolation between points.
 - **exp_up:** Concave-upward exponential growth (slow start, rapid finish), with curvature controlled by $\alpha = 3.0$.
 - **exp_down:** Concave-downward exponential curve (rapid start, slow finish), the complement of **exp_up**.
 - **bell:** Cosine envelope ($\frac{1}{2} - \frac{1}{2} \cos(\pi x)$), generating a smooth S-curve suitable for Gaussian-like bell shapes.

4 1D Convolutional Neural Networks: Theoretical Background

This section provides a comprehensive, ground-up theoretical foundation for 1D Convolutional Neural Networks (1D-CNNs). It is designed to provide sufficient mathematical rigor so that a reader could implement a 1D-CNN from scratch in C or NumPy, without relying on high-level libraries like PyTorch.

4.1 The 1D Convolution Operation (Forward Pass)

Standard fully-connected networks are ill-suited for time-series signal processing. They treat the input as an unordered set of independent variables, destroying temporal topology, and they lack **translation equivariance** (the ability to recognize a pattern regardless of where it occurs in time).

A 1D-CNN solves this by using a "sliding window" approach. Instead of learning a massive weight matrix for the entire signal, the network learns small templates called **filters** or **kernels**.

4.1.1 Single Channel Convolution

Let $\mathbf{x}$ be a 1D input signal of length T , and $\mathbf{w}$ be a learnable kernel of size K . The convolution operation slides the kernel $\mathbf{w}$ across the signal $\mathbf{x}$. At each time step t , it computes the dot product between the kernel and the local patch of the signal:

$$y[t] = \sum_{i=0}^{K-1} w[i] \cdot x[t+i] + b \quad (1)$$

where b is a learnable bias scalar. Mathematically, this dot product acts as a **pattern matcher**. If the signal patch $x[t \dots t+K-1]$ is perfectly aligned with the weights in the kernel w , the output $y[t]$ produces a massive positive activation. The resulting sequence $\mathbf{y}$ is called a **feature map**.

4.1.2 Multi-Channel Convolution

A single filter can only detect one specific shape (e.g., a rising edge). To detect complex anomalies, a layer must learn dozens of filters simultaneously. Furthermore, deeper layers in the network do not operate on the raw 1D signal; they operate on the multiple feature maps generated by the previous layer.

Let C_{in} be the number of input channels and C_{out} be the number of desired output channels (filters). The kernel is now a 3D tensor $\mathbf{W}$ of shape (C_{out}, C_{in}, K) . The output feature map for a specific output channel c_{out} at time t is the sum of convolutions across all input channels c_{in} :

$$y_{c_{out}}[t] = \sum_{c_{in}=0}^{C_{in}-1} \sum_{i=0}^{K-1} W[c_{out}, c_{in}, i] \cdot x_{c_{in}}[t+i] + b_{c_{out}} \quad (2)$$

This operation preserves **local connectivity** (neurons only look at a small temporal patch K) and enforces **weight sharing** (the exact same tensor $\mathbf{W}$ is applied at every time step t), which drastically prevents overfitting and reduces computational cost.

4.2 Non-linear Activations

Without a non-linear activation function, stacking multiple convolutional layers would mathematically collapse into a single, linear convolution. After computing the feature map $\mathbf{y}$, we apply an activation function σ element-wise.

4.2.1 ReLU (Rectified Linear Unit)

The most widely used activation in CNN hidden layers is ReLU. It passes positive activations (where the filter "found" its target pattern) and zeroes out negative ones:

$$a[t] = \max(0, y[t]) \quad (3)$$

Its derivative is trivially 1 if $y[t] > 0$, and 0 otherwise. This simplicity prevents the vanishing gradient problem during deep backpropagation.

4.2.2 Softmax

At the very end of the network, after flattening the final feature maps into a vector of logits $\mathbf{z}$ (one for each class), we apply the Softmax function to convert them into a valid probability distribution:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_k e^{z_k}} \quad (4)$$

4.3 Pooling Layers (Downsampling)

To hierarchically build larger receptive fields and reduce computational cost, CNNs use **Pooling layers** between convolutions.

MaxPooling slides a small window of size P (usually 2) across the feature map with a stride of P . Instead of a dot product, it simply outputs the maximum value in that window:

$$p[t] = \max_{i \in [P \cdot t, P \cdot t + P - 1]} a[i] \quad (5)$$

Intuitively, MaxPooling asks: *"Did this specific feature appear anywhere within this small region?"* If yes, it keeps the high activation and discards the exact spatial coordinate. This halves the length of the signal ($T \rightarrow T/2$) and provides **local translation invariance** against small signal jitters.

Crucially for implementation, during the forward pass, a MaxPooling layer must cache the **argmax index** (the exact position i that contained the maximum value) because gradients will only be routed back through that specific winning "pixel" during back-propagation.

4.4 The Loss Function

To train the CNN, we quantify its prediction error using **Cross-Entropy Loss**. For a single window, comparing the true one-hot label vector $\mathbf{y}$ against the predicted probabilities $\hat{\mathbf{y}}$:

$$\mathcal{L} = - \sum_k y_k \log(\hat{y}_k) \quad (6)$$

Proof: Derivative of Softmax with Cross-Entropy

A fundamental result in machine learning is how elegantly the gradient simplifies at the output layer. We want to find the error signal $\delta_j = \frac{\partial \mathcal{L}}{\partial z_j}$ where z_j is the pre-softmax logit. By the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \sum_k \frac{\partial \mathcal{L}}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_j} \quad (7)$$

1. Derivative of Loss: $\frac{\partial \mathcal{L}}{\partial \hat{y}_k} = -\frac{y_k}{\hat{y}_k}$ 2. Derivative of Softmax (quotient rule): - If $k = j$: $\hat{y}_j(1 - \hat{y}_j)$ - If $k \neq j$: $-\hat{y}_k \hat{y}_j$

Substituting these into the sum:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z_j} &= \left(-\frac{y_j}{\hat{y}_j} \right) (\hat{y}_j(1 - \hat{y}_j)) + \sum_{k \neq j} \left(-\frac{y_k}{\hat{y}_k} \right) (-\hat{y}_k \hat{y}_j) \\ &= -y_j(1 - \hat{y}_j) + \sum_{k \neq j} y_k \hat{y}_j = -y_j + y_j \hat{y}_j + \hat{y}_j \sum_{k \neq j} y_k \end{aligned}$$

Since the true labels y must sum to 1, $\sum_{k \neq j} y_k = 1 - y_j$. Substituting this:

$$= -y_j + y_j \hat{y}_j + \hat{y}_j(1 - y_j) = -y_j + y_j \hat{y}_j + \hat{y}_j - y_j \hat{y}_j$$

The complex terms cancel out, leaving the beautiful final result for the output layer error:

$$\delta_j^{\text{out}} = \frac{\partial \mathcal{L}}{\partial z_j} = \hat{y}_j - y_j \quad (8)$$

4.5 Backpropagation in 1D-CNNs (Backward Pass)

Once the output error δ^{out} is calculated, the **Backpropagation algorithm** uses the chain rule to pass this error backward through the network, layer by layer, to compute the gradient of the loss with respect to every single filter weight.

4.5.1 Backpropagating through MaxPooling

Because MaxPooling only routed the maximum value forward, it acts as a gate. The error δ from the subsequent layer is routed entirely to the specific index that "won" the max pooling (which we cached during the forward pass). All other indices in the pooling window receive a gradient of 0, because changing them slightly would not have affected the forward output.

4.5.2 Backpropagating through 1D Convolution

This is the mathematical core of training a CNN. Let the error signal received from the next layer be $\delta[t] = \frac{\partial \mathcal{L}}{\partial y[t]}$. We must compute two sets of gradients for our single-channel convolution equation (Eq. 1).

1. Gradient with respect to the filter weights ($w[i]$): A single weight $w[i]$ in the filter was multiplied by the input $x[t + i]$ at *every* time step t as the filter slid across the signal. By the chain rule, we sum its contribution over all time steps:

$$\frac{\partial \mathcal{L}}{\partial w[i]} = \sum_t \frac{\partial \mathcal{L}}{\partial y[t]} \frac{\partial y[t]}{\partial w[i]} = \sum_t \delta[t] \cdot x[t + i] \quad (9)$$

This reveals a profound mathematical symmetry: to find how to update the filter weights, we perform a **cross-correlation** between the forward input signal x and the backward error signal δ . (For multi-channel convolutions, this cross-correlation is summed across all batch items and input dimensions).

2. Gradient with respect to the input ($x[t]$): To pass the error further backward to earlier layers, we need $\frac{\partial \mathcal{L}}{\partial x[t]}$. A single input point $x[t]$ affects multiple outputs y as the kernel slides over it. Specifically, it is multiplied by $w[0]$ to create $y[t]$, $w[1]$ to create $y[t - 1]$, and so on up to $w[K - 1]$ to create $y[t - (K - 1)]$.

Summing these contributions via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial x[t]} = \sum_{i=0}^{K-1} \frac{\partial \mathcal{L}}{\partial y[t-i]} \frac{\partial y[t-i]}{\partial x[t]} = \sum_{i=0}^{K-1} \delta[t-i] \cdot w[i] \quad (10)$$

This is an elegant symmetry: to pass the error backward, we perform a **full convolution** of the error signal δ with the **flipped** kernel w . This allows deep learning frameworks to reuse highly optimized forward convolution algorithms for the backward pass.

4.6 Optimization and Regularization

4.6.1 Adam Optimizer

With the gradient of the weights $\nabla_{\mathbf{w}} \mathcal{L}$ computed, we update the filters to minimize the loss. Instead of pure Stochastic Gradient Descent ($\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$), we use the **Adam** optimizer. Adam maintains exponential moving averages of both the gradients (momentum) and the squared gradients (scaling). This provides an adaptive, per-parameter learning rate that navigates flat plateaus and steep ravines in the loss landscape much faster than standard SGD.

4.6.2 Regularization: Dropout and BatchNorm

To prevent the CNN filters from memorizing the specific noise patterns of the training data (overfitting), two mechanisms are used:

- **Dropout:** Randomly zeroes out a percentage of the activations in the fully connected classification head during training. This prevents the network from over-relying on any single feature detector.
- **Batch Normalization:** Inserted immediately after convolutional layers, it normalizes the output feature maps to have zero mean and unit variance across the mini-batch. This stabilizes the distribution of inputs to the next layer, allowing for higher learning rates and significantly faster convergence.

5 1D-CNN Implementation

This section describes the concrete PyTorch implementation of the 1D-CNN for multi-class anomaly classification. The codebase is organized into modular scripts within the `1D-CNN/` directory, with full CUDA GPU acceleration support.

5.1 Data Processing (`train.py`)

The data loading and preprocessing logic is embedded in the `load_data()` function within `train.py`. It bridges the synthetic `.csv` generator and the neural network through the following steps:

1. **Temporal Split:** The continuous signal is split chronologically *before* windowing: the first 80% of time steps are reserved for training and the remaining 20% for testing. This prevents **data leakage**, which would occur with a random split because overlapping windows (stride 32 over a 512-point window) share up to 94% of their data points.
2. **Sliding Window Extraction:** Each split is independently sliced into overlapping windows (size 512, stride 32).
3. **Threshold-Based Labeling:** A window is only labeled as a specific anomaly class if the anomaly occupies at least **10%** of the window length. This prevents the CNN from learning that windows containing 99% pure noise should be classified as anomalies. When the threshold is met, the most frequent anomaly class ID within the window is assigned via majority voting.
4. **Per-Window Centering (Instance Normalization):** To make the network robust against the slowly wandering background baseline generated by the Ornstein-Uhlenbeck process, each sliding window is independently centered by subtracting its own local mean μ_{window} . However, a global standard deviation σ_{train} is computed across all centered training windows to preserve the relative amplitudes of the spikes. The test set is scaled using this same global statistic. This value is persisted to `normalization_stats.json` for identical inference normalization:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{window}}}{\sigma_{\text{train}}} \quad (11)$$

5. **Tensor Reshaping:** The output is reshaped into the strict `(Batch_Size, 1, Window_Size)` format required by PyTorch's `Conv1d` layers.

5.2 Architecture Definition (`model.py`)

The `Anomaly1DCNN` class inherits from `torch.nn.Module` and implements a 3-block convolutional architecture with a progressively narrowing kernel strategy:

- **Block 1:** `Conv1d(in=1, out=16, kernel=7, padding=3) → BatchNorm1d → ReLU → MaxPool1d(2)`.
- **Block 2:** `Conv1d(in=16, out=32, kernel=7, padding=3) → BatchNorm1d → ReLU → MaxPool1d(2)`.

- **Block 3:** `Conv1d(in=32, out=64, kernel=5, padding=2) → BatchNorm1d → ReLU → MaxPool1d(2)`.

The early layers use larger kernels (7) to capture broad temporal structure such as the overall envelope of anomaly shapes, while the final layer uses a smaller kernel (5) to refine local features. The channel count doubles at each block ($16 \rightarrow 32 \rightarrow 64$), progressively expanding the network’s feature representation capacity.

The network utilizes a **Regional Pooling layer** (`AdaptiveAvgPool1d(4)`) instead of Global Average Pooling. This preserves 4 distinct temporal bins, allowing the network to recognize the temporal topography of shapes—for example, distinguishing the two separate peaks of an M shape from a solid `bell_sq` block, since the averaged values in each bin differ between these shapes.

This flattened vector of size 256 (64×4) is then passed through a classifier head: `Flatten() → Dropout(0.4) → Linear(256, 32) → ReLU → Dropout(0.2) → Linear(32, 7)`. The total model footprint is **22,727 trainable parameters**.

5.3 Training Loop (`train.py`)

The training script establishes the complete PyTorch training ecosystem with GPU acceleration and comprehensive logging:

- **Data Split:** The training set is further divided 80/20 into train/validation subsets using a random permutation for early stopping monitoring.
- **Class Imbalance Handling:** Computes inverse frequency weights $w_c = \frac{N_{\text{total}}}{C \cdot N_c}$ and passes them to the `CrossEntropyLoss` criterion, forcing the optimizer to treat rare anomaly shapes with the same importance as the dominant normal background.
- **Optimization:** Adam optimizer with initial learning rate 10^{-3} .
- **Learning Rate Scheduling:** `ReduceLROnPlateau` (patience 2, factor 0.5) monitors validation loss and halves the learning rate when progress stalls.
- **Early Stopping:** Patience of 5 epochs. The best model weights are saved only when validation loss improves.
- **Logging:** A comprehensive `.txt` log and an interactive Plotly training curves chart (HTML + PNG) are generated.

5.3.1 Training Results

The model was trained on a 10M-point synthetic dataset (249,985 windows) on an NVIDIA GeForce RTX 5060 Laptop GPU with CUDA 12.8:

- **Convergence:** 82 epochs, 9 minutes 5 seconds. Learning rate decayed from 10^{-3} to 8×10^{-6} across 7 reduction steps.
- **Final losses:** Best validation loss = 0.2494, test loss = 0.3028.
- **Test accuracy:** 93.4% per-window accuracy (62,485 windows). Event-level accuracy is significantly higher due to overlapping window accumulation during inference.

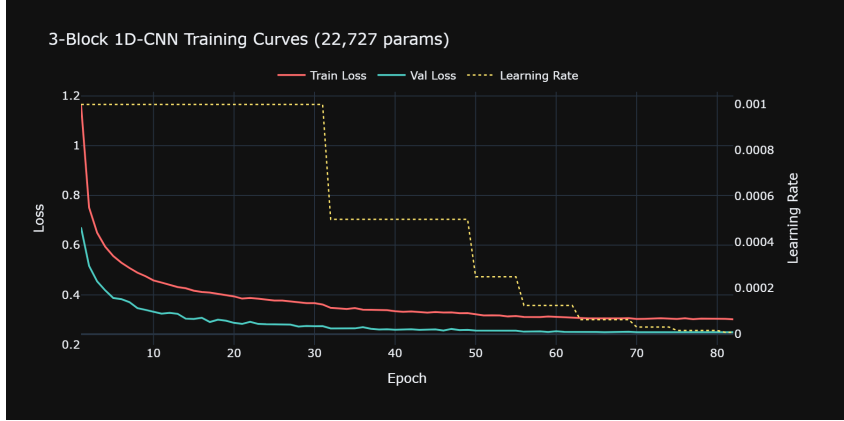


Figure 4: Training and validation loss curves with learning rate schedule. The validation loss remains below the training loss due to class-weighted cross-entropy inflating the training loss for minority anomaly classes.

5.4 Architecture Search

The final architecture was selected through an empirical search over several candidate configurations. All variants were trained on the same 10M-point dataset and evaluated on both per-window test accuracy and event-level inference over a 100,000-point segment containing 77 anomaly events:

- **Depth vs. Width:** Reducing the number of convolutional blocks below 3 degrades performance unless channel width is increased. A 2-block model with narrow channels (16→32) lacked sufficient representational capacity, while a 2-block model with wider channels (32→64) recovered most accuracy.
- **Diminishing returns of depth:** Blocks 4 and 5 contributed negligible accuracy gains while inflating the parameter count by 12 $\times$.
- **Training speed:** Tightening early stopping patience from 10 to 5 and LR scheduler patience from 4 to 2, combined with doubling the batch size to 128, reduced training time without affecting convergence.

Table 1: Comparison of 1D-CNN architecture variants. Event-level metrics are computed over 77 anomaly events in a 100K-point inference segment. All models use `AdaptiveAvgPool1d(4)` regional pooling.

Variant	Channels	Params	Train Time	Event Acc.	Mean Conf.
2-block narrow	16-32	3,719	8 min 12 s	88.3%	66.8%
2-block wide	32-64	19,207	5 min 49 s	98.7%	79.7%
3-block	16-32-64	22,727	5 min 45 s	100%	90.2%
4-block	16-32-64-64	35,207	7 min 18 s	100%	90.2%
5-block	32-64-128-128-256	270,725	7 min 58 s	100%	91.9%

The 3-block architecture was selected as the final model, achieving perfect event-level classification with 12 $\times$ fewer parameters than the 5-block architecture and the fastest training time.

5.5 Inference and Evaluation (visualize_inference.py)

The inference script evaluates the trained model on a continuous segment of the dataset using a fine-grained sliding window (stride 10, window 512). Two key design decisions:

1. **Consistent Normalization:** The inference script loads the global σ_{train} from `normalization_stats.json` and applies the same per-window centering strategy.
2. **Overlapping Window Accumulation:** The softmax probability vector is **accumulated** across all points covered by each window. With stride 10 and window 512, each point is covered by up to 51 overlapping windows. The final per-point class probability is the average:

$$P_{\text{final}}(c \mid t) = \frac{1}{|\mathcal{W}_t|} \sum_{w \in \mathcal{W}_t} P_w(c) \quad (12)$$

where $\mathcal{W}_t$ is the set of all windows covering time step t . This ensemble-like averaging significantly reduces prediction noise.

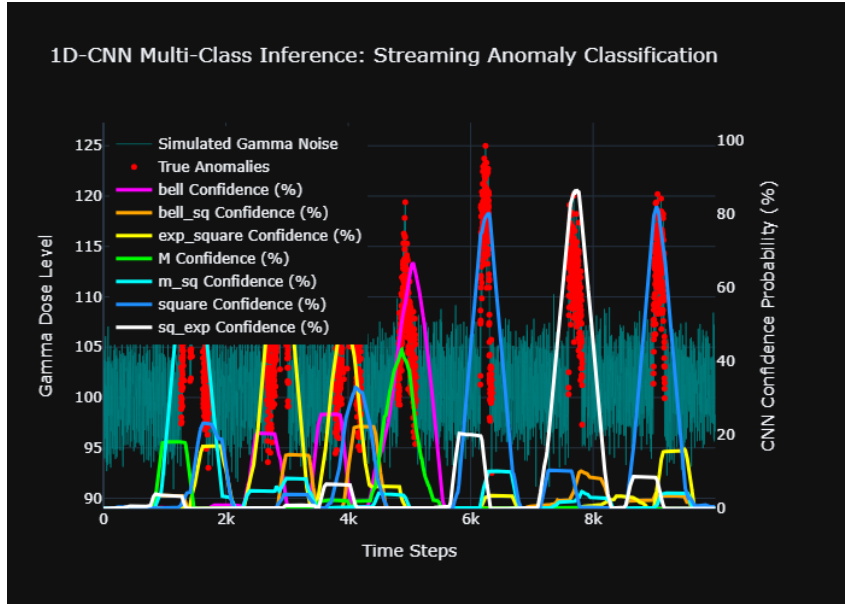


Figure 5: Multi-Class 1D-CNN Confidence Probability mapped against Simulated Gamma Noise

5.5.1 Inference Results

The model was evaluated on a 1,000,000-point segment containing 777 anomaly events:

Table 2: Event-level inference results over 777 anomaly events. Detection rate indicates whether the model identified the region as any anomaly type; exact classification requires the correct class.

Metric	Value
Total events	777
Detection rate (any anomaly)	100.0% (777/777)
Exact classification accuracy	98.3% (764/777)
Mean confidence	88.8%
Inference throughput	1,421 windows/sec
Inference time (1M points)	70 s

Table 3: Per-class Precision, Recall, and F1-Score at event-level granularity.

Class	Support	Precision	Recall	F1-Score
bell	128	0.992	0.992	0.992
bell_sq	136	0.963	0.956	0.959
fast_ascend	129	1.000	0.992	0.996
fast_descend	116	0.983	0.991	0.987
M	126	0.962	0.992	0.977
square	142	1.000	0.979	0.989
Macro Average		0.983	0.984	0.984

The 13 misclassifications are concentrated in the `bell_sq` class (6 errors), which is the most morphologically ambiguous shape—its flat-topped profile can be confused with `square`, and its rounded shoulders can resemble `M` at high deformation levels.

5.6 Robustness Analysis (`robustness_test.py`)

To assess the model’s sensitivity to degraded signal conditions, a robustness test independently sweeps two axes while holding the other at its training baseline:

1. **Background noise intensity:** The standard deviation of the high-frequency sensor noise was scaled from $1\times$ to $3\times$ the baseline value ($\sigma = 2.70$ gamma dose units).
2. **Anomaly shape deformation:** The variance parameter of the anomaly generator was scaled from $1\times$ to $8\times$ the baseline value (0.04), controlling how much each anomaly instance deviates from its canonical template shape.

Each configuration generated a test dataset with 50 anomaly events using a fixed random seed to ensure identical anomaly positions and template selections across all runs.

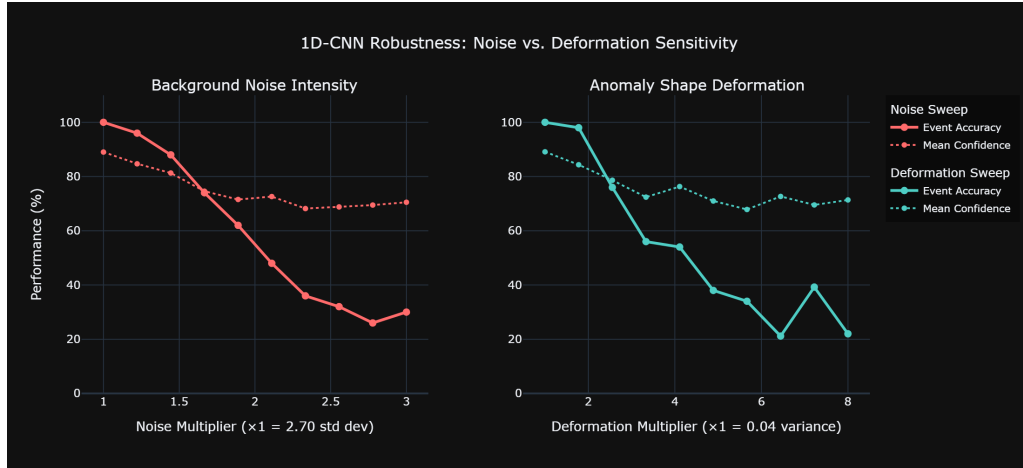


Figure 6: Model robustness under increasing noise (left) and shape deformation (right). Event accuracy (solid) and mean confidence (dashed) are shown. The model is substantially more sensitive to shape deformation than to background noise.